Infrastructure as Code

if it isn't in Git, it doesn't exist

Watch the 4-min video

> **In one line:** describe infrastructure declaratively in versioned files, review a **plan** before every **apply**, and guard the **state** that maps code to real resources (remote, encrypted, locked, split); reuse through versioned modules and catch **drift** by planning regularly.

1 Built by clicking

Production was built in the cloud portal two years ago. Now you need an identical copy in a new region. Nobody remembers every setting, half the decisions were never written down, and someone changed a firewall rule by hand last month. With **infrastructure as code** (IaC), networks, clusters, databases and permissions live in files that are reviewed, versioned and can recreate the same environment again and again: for a new region, for disaster recovery, or for a short-lived test environment.

2 Declarative tools

Declarative tools take the **desired end state** and work out the create/update/delete calls themselves, so running them twice changes nothing the second time (they are idempotent). Imperative scripts (a series of CLI calls) break on the second run or when something already exists.

Tool	Language	Where state lives	Notes
Terraform / OpenTofu	HCL	a state file you store	multi-cloud, huge provider ecosystem; OpenTofu is the open-source fork after the 2023 licence change
Bicep / ARM	Bicep DSL	Azure Resource Manager itself	Azure only; no state file; `what-if` previews changes
CloudFormation / CDK	YAML/JSON, or code via CDK	AWS stacks	AWS only; change sets preview; built-in drift detection
Pulumi	TypeScript, Python, Go, C#	Pulumi Cloud or your bucket	real programming languages; same plan/apply model (`preview` / `up`)

3 Plan, then apply

`terraform plan` refreshes the real resources, compares them with the code and the state, and prints exactly what will be **created (+)**, **changed in place (~)**, **destroyed (−)** or **replaced (−/+)**. `apply` executes it. Save the plan (`plan -out=tfplan` , then `apply tfplan`) so what runs is exactly what was reviewed, not a fresh plan computed minutes later against a changed world.

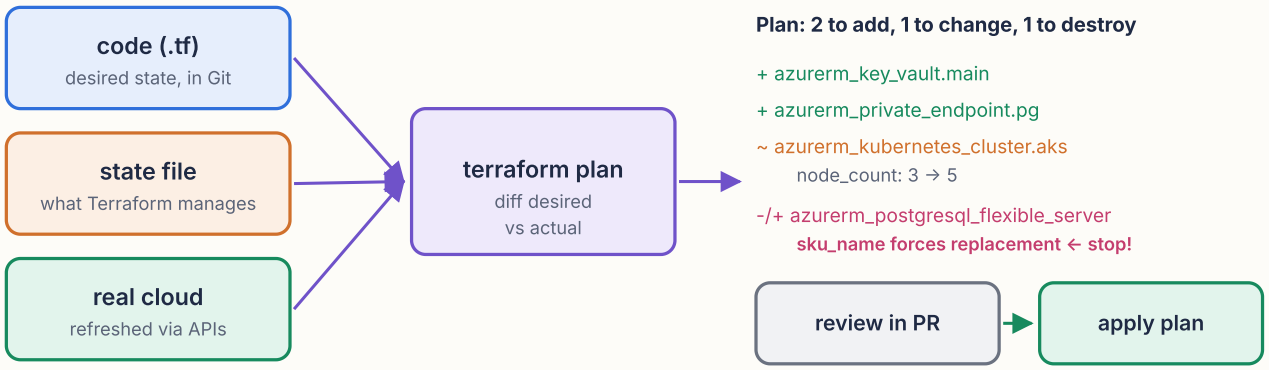
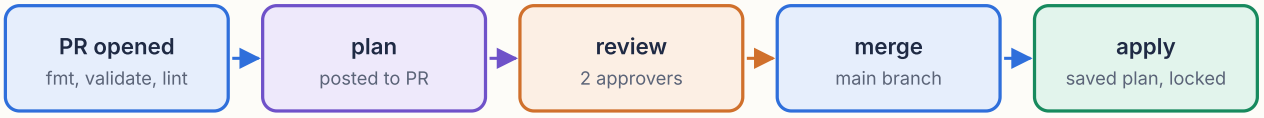


Figure 1. A plan is a three-way comparison. The line to catch in review is **−/+**: a changed attribute that forces the database to be destroyed and recreated.

Reviewing the plan in the pull request catches the expensive mistakes: a renamed resource that Terraform will delete and recreate (fix with a `moved` block), an attribute that forces replacement of a database, or a destroy you didn't expect. For critical resources add `lifecycle { prevent_destroy = true }`.



Policy checks (OPA / Sentinel / Checkov) and cost estimates (Infracost) can run at the plan step.

Figure 2. A typical IaC pipeline. CI holds the cloud credentials (ideally via workload identity, no stored secrets); humans never apply from laptops.

4 State: the map from code to reality

Terraform's **state** is a JSON document mapping every resource address in your code (`module.aks.azurerm_kubernetes_cluster.main`) to a real cloud ID and its last known attributes. It's how Terraform knows that it owns a resource, what to change and in which order to delete. Three things go wrong:

- **Lost state:** Terraform no longer knows it manages anything. The next apply tries to create everything again and collides with existing names; recovery means importing every resource by hand.
- **Concurrent applies:** two runs read the same state, both create or both modify, and the last writer's state wins, leaving duplicates or orphans.
- **Secrets inside:** attributes such as generated database passwords and access keys are stored in state in plain text. Whoever can read the state can read them.

5 Protecting state

- **Remote backend**: an Azure storage account or S3 bucket, with encryption at rest and **versioning** (soft delete) so you can roll back a corrupted state.
- **Locking**: only one apply at a time. The azurerm backend uses a blob lease; the S3 backend now supports a native lock file (older setups use a DynamoDB table).
- **Tight access**: only the pipeline's identity can write; few humans can read. Treat the bucket like a secrets store.
- **No hand edits**: use `terraform state mv`, `import` and `moved` blocks, which are reviewed and reversible.

- **Split states** to limit the blast radius: by environment and by layer, so a bad app change can't plan a VNet deletion and plans stay fast.

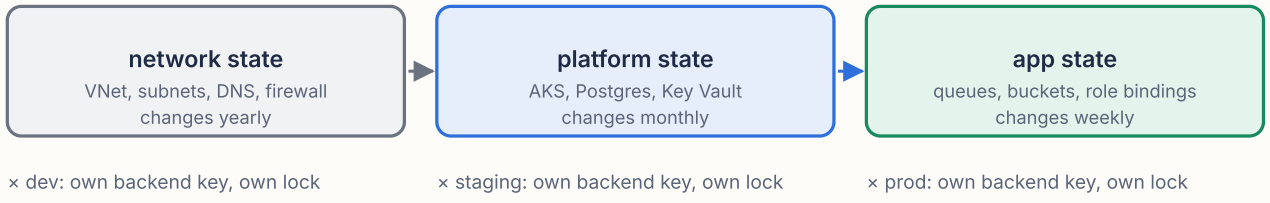


Figure 3. Split state by layer and environment. Each box is a separate backend key, lock and pipeline.

6 Modules and drift

Modules package a reusable unit (a network, an AKS cluster with standard settings) with input variables and outputs. Pin module versions with a Git tag or registry version, so upgrading a module is a deliberate, reviewed change for each environment. Keep environments thin: the same modules, different variable files.

Drift is any change made outside the code: a portal click, a hot-fix, another tool. Detect it with a scheduled `terraform plan -detailed-exitcode` (exit code 2 means changes are pending) or `plan -refresh-only`, and alert. Fix it either way round: bring the change into code if it was right, or apply to put things back if it wasn't. Restricting write access in production to the pipeline is what makes drift rare.

```
terraform {
  backend "azurerm" {                # remote, encrypted, versioned
    resource_group_name  = "tfstate-rg"
    storage_account_name = "tfstateprod01"
    container_name       = "tfstate"
    key                  = "platform/prod.tfstate"   # lock = blob lease
  }
}
module "aks" {
  source     = "git::https://git.example.com/tf-modules//aks?ref=v2.3.0"
  region     = var.region           # only variables differ per region
  node_count = var.aks_nodes
  subnet_id  = module.network.aks_subnet_id
}
```

Figure 4. Remote state with locking, and a version-pinned module whose only per-region differences are variables.

WORKED EXAMPLE: AKS, POSTGRES AND KEY VAULT WITH TERRAFORM

Modules define the **network** (VNet, subnets, private DNS zones), an **AKS** cluster, an **Azure Database for PostgreSQL** flexible server reachable only through a private endpoint, and a **Key Vault** for secrets. Each environment's state lives in an Azure storage account with versioning, blob-lease locking and access limited to the pipeline identity. A pull request runs `plan` and posts the output; reviewers check for replacements and destroys; after approval and merge the pipeline applies the saved plan. For the new region, the team adds `westeurope.tfvars` (region, CIDR ranges, node counts) and a new backend key, then runs the same code. The database password is generated and written to Key Vault, so applications never see it in Git, only in state, which is why state access is restricted.

COMMON MISTAKES

- Local state on a laptop, or state committed to Git with secrets inside.
- Applying without reading the plan, missing a forced replacement of a database.
- One giant state for everything: slow plans and a huge blast radius.
- Unpinned modules and providers, so an upstream release changes production unannounced.

7 Key terms

Declarative: describe the end state; the tool computes the steps

Plan / apply: preview the diff / execute it

State: mapping of code addresses to real resource IDs and attributes

State locking: only one run may modify state at a time

Module: reusable, versioned, parameterised group of resources

Drift: real infrastructure differs from code

8 Interview questions

Q1. Why is Terraform state important and how do you protect it? BASIC

It maps code to real resources so Terraform knows what it manages and what to change; protect it with remote, encrypted, versioned storage, locking, restricted access, no hand edits, and split states to limit blast radius.

Q2. What is drift? BASIC

Changes made to infrastructure outside the code; detect it with regular plans and fix it by reconciling, either updating the code or re-applying.

Q3. Why declarative rather than a script of CLI calls? BASIC

Declarative tools compute the difference from current state, so they are idempotent, show a reviewable plan, and can recreate or update environments safely.

Q4. The plan shows your production database will be replaced. What do you do? SCENARIO

Stop; find the attribute forcing replacement or the renamed address; use a moved block or revert the change, or plan a controlled migration; add prevent_destroy to protect it.

Q5. How do you build an identical environment in a new region? SCENARIO

Use the same version-pinned modules with a new variables file and a new state backend key, run plan in a PR, review, and apply through the pipeline.

Q6. Two engineers ran apply at the same time and you now have duplicate resources. What was missing and how do you clean up? TRICKY

State locking and a single apply path; clean up by importing or removing the orphans with state commands, then enforce locking and pipeline-only applies.

Q7. Why can Bicep work without a state file when Terraform can't? TRICKY

Bicep deploys to Azure Resource Manager, which itself knows every resource in a deployment scope; Terraform is cloud-agnostic and must keep its own record of what it created.

Next: SD-V-012 · GitOps backendroadmap.com